

Zecpath AI Job Portal

Day 16 — ATS API Design & Integration Planning

Objective: make the ATS AI pipeline (Days 5–14) consumable by backend systems through a well-defined REST API with async job handling, consistent error semantics, and a logging standard suitable for production operation.

1. Scope & Design Principles

- Resource-oriented REST API versioned at /api/v1, JSON over HTTPS.
- Every compute-heavy pipeline stage (resume parsing, matching, shortlisting) is exposed as an async Task — one reusable Task model across all operations, not a bespoke status field per endpoint.
- Request/response contracts map 1:1 onto existing pipeline data classes (ExtractionRecord, JobRequirementRecord, ATS Scoring Engine output, Day 14 ranking output) so the API is a thin, additive layer — consistent with the project's additive-only convention.
- The Day 7 metadata envelope (schema_version, model_version, pipeline_version) is echoed on every stored resource in API responses.
- Errors follow RFC 7807 (problem+json) extended with a stable machine-readable code and a trace_id for log correlation.
- Idempotency-Key support on all resource-creating POSTs, since network retries are expected from backend integrators.

2. Resource Model

Four primary resources, plus a generic Task resource used for polling any async operation:

Resource	Purpose	Backing pipeline stage(s)
Resume	Uploaded file + all downstream parsing enrichment	Day 5, 8, 9, 10, 11
JobDescription	Parsed job requirement record	Day 6
Match	Single resume × JD score with explainability	Day 12, 13
Shortlist	Batch match + ranked/zoned candidate list	Day 13, 14
Task	Generic async job status/result pointer	cross-cutting

3. Endpoint Summary

Method & Path	Purpose	Sync/Async
POST /resumes	Upload a resume file	Async (202)
GET /resumes/{id}	Fetch resume + enrichment fields	Sync
POST /job-descriptions	Submit JD text for parsing	Sync (201) / Async (202) for large text
GET /job-descriptions/{id}	Fetch parsed JD	Sync
POST /matches	Score one resume against one JD	Async (202)
GET /matches/{id}	Fetch match result	Sync
POST /job-descriptions/{id}/shortlist	Batch-score + rank resumes for a JD	Async (202)
GET /shortlists/{id}	Fetch ranked shortlist	Sync

GET /shortlists/{id}/export	Download recruiter CSV (Day 14 format)	Sync
GET /tasks/{id}	Poll status/result of any async operation	Sync

Full request/response schemas are defined in the accompanying `openapi.yaml` and validated as OpenAPI 3.0.3. Standalone JSON Schema (draft-07) files are provided per-contract under `/schemas` for consumers that prefer not to parse the whole OpenAPI document.

4. Async Job Handling Design

4.1 Why async

Resume file parsing (OCR fallback, multi-column PDF reconstruction) and semantic matching (sentence-transformers inference) are the two most latency-variable stages in the pipeline. Rather than holding an HTTP connection open with unpredictable duration, resume upload, matching, and shortlisting all return 202 Accepted immediately with a Task reference.

4.2 Task lifecycle

State	Meaning
queued	Accepted, waiting for a worker
processing	A worker has picked it up; progress may update
completed	result_url is populated; fetch the resource normally
partial	Result available but with warnings (mirrors the existing partial status used by the Day 5 extraction engine and Day 13 scoring engine)
failed	error is populated with a full ErrorResponse body

4.3 Consumption patterns

- Polling: GET /tasks/{task_id}. The response carries a Retry-After header (seconds) while queued/processing; clients must respect it rather than tight-polling.
- Webhooks: an optional callback_url on the creating request. On completion the same Task body is POSTed to that URL, signed with an HMAC header (X-Zecpath-Signature) so integrators can verify authenticity.
- Idempotency: an Idempotency-Key header on POST /resumes, /job-descriptions, /matches, and /shortlist. A retried request with the same key within 24h returns the original Task (200) instead of re-enqueuing work; a reused key with a different body returns 409 IDEMPOTENCY_KEY_REUSED.
- Batch shortlisting fans out internally into per-resume match sub-tasks; the parent Task's progress reflects completed/total sub-tasks so large batches (hundreds of resumes) give callers a meaningful progress signal instead of an opaque single state flip.

5. Error Handling Standard

All error responses use Content-Type: application/problem+json (RFC 7807) with two additions used consistently across the whole API: a stable code enum for programmatic branching, and a trace_id that a caller can hand to support and that appears verbatim in server logs.

HTTP Status	code	When
400	VALIDATION_ERROR	Malformed body, missing required field, weights not summing to 1.0
404	RESOURCE_NOT_FOUND	Resume/JD/Match/Shortlist/Task id does not exist

409	IDEMPOTENCY_KEY_REUSED	Same Idempotency-Key, different request body
413	PAYLOAD_TOO_LARGE	Resume file exceeds the configured size limit
415	UNSUPPORTED_FILE_TYPE	File extension outside { .pdf, .docx } — mirrors UnsupportedFileTypeError from the Day 5 engine
422	PARSING_FAILED	File accepted but extraction/parsing produced no usable text (status=failed on the underlying record)
429	RATE_LIMITED	Caller exceeded their rate limit; Retry-After present
503	DEPENDENCY_UNAVAILABLE	Semantic matching model unavailable; caller should expect the pipeline to fall back to the Day 12 offline hashing matcher rather than fail outright
500	INTERNAL_ERROR	Unhandled server fault

Field-level validation failures additionally populate an errors array of {field, message} pairs so client-side forms can highlight the offending input directly.

6. Logging Standard

- Every request/response pair is logged once as a single structured JSON line: timestamp, X-Request-ID (= trace_id), method, path, status_code, duration_ms, and, where applicable, resume_id / job_description_id / match_id / shortlist_id / task_id.
- Log levels reuse the existing utils/logger.py convention: INFO for normal milestones (task enqueued, task completed), WARNING for recoverable issues (partial extraction, fallback matcher engaged), ERROR for failed tasks, CRITICAL reserved for process-level faults.
- PII / content redaction: raw resume text, cleaned_text, and narrative_explanation are never written to logs — only resource ids, char counts, and status fields. This mirrors the project's existing practice of keeping cleaned text in structured JSON output rather than log files.
- X-Request-ID is accepted from the caller (for cross-system trace correlation) or generated server-side if absent, and is echoed back on every response, including error responses, as trace_id.
- Async worker logs additionally record queue wait time and processing time as separate fields, so p95/p99 latency per Task type can be tracked independently of HTTP-layer latency.

7. Versioning & Compatibility

- URI versioning (/api/v1/...); breaking changes ship as /api/v2 rather than mutating v1 contracts, consistent with the project's additive-only convention already used for pipeline output schemas.
- New optional fields may be added to any response at any time without a version bump; consumers must ignore unknown fields.
- The schema_version inside the metadata envelope tracks the underlying record schema independently of the API URI version, so a v1 API can serve records produced by a newer pipeline schema_version without a breaking change.

8. Deliverables in this Package

- openapi.yaml — OpenAPI 3.0.3 specification (validated with openapi-spec-validator).
- schemas/*.schema.json — eight standalone JSON Schema (draft-07) files for the core request/response contracts, self-contained via inlined \$defs.

- `examples/*.json` — sample payloads for a processing Task, a completed MatchResult, and an ErrorResponse, each validated against its schema.
- This document — narrative design rationale for async job handling, error, and logging standards.

Appendix A — Task Object (Reference)

```
{
  "task_id": "uuid",
  "type": "resume_extraction | jd_parsing | match | shortlist",
  "status": "queued | processing | completed | partial | failed",
  "progress": 0-100 | null,
  "created_at": "ISO-8601",
  "updated_at": "ISO-8601",
  "result_url": "https://.../resumes/{id}" | null,
  "error": { ...ErrorResponse } | null
}
```

Appendix B — Error Response (Reference)

```
{
  "type": "https://api.zecpath.example.com/errors/unsupported-file-type",
  "title": "Unsupported file type",
  "status": 415,
  "detail": "File extension '.rtf' is not supported. Supported
types: .pdf, .docx",
  "instance": "/api/v1/resumes",
  "code": "UNSUPPORTED_FILE_TYPE",
  "trace_id": "req_9f1c2e7a4b3d"
}
```